dependencies into a service in other ways, such as by using property injection, but these techniques aren't supported by the built-in DI container.

I hope that this example made the advantages of using DI in your code apparent, but in many ways these benefits are secondary to the main benefit of using DI. In particular, DI helps keep your code loosely coupled by coding to interfaces.

8.2 Creating loosely coupled code

Coupling is an important concept in object-oriented programming, referring to how a given class depends on other classes to perform its function. Loosely coupled code doesn't need to know a lot of details about a particular component to use it.

The initial example of `RegisterUser` and `EmailSender` was an example of tight coupling; you were creating the `EmailSender` object directly and needed to know exactly how to wire it up. On top of that, the code was difficult to test. Any attempts to test `RegisterUser` would result in an email being sent. If you were testing the controller with a suite of unit tests, that approach would be a surefire way to get your email server blacklisted for spam!

Taking `EmailSender` as a parameter and removing the responsibility of creating the object helps reduce the coupling in the system. If the `EmailSender` implementation changes so that it has another dependency, you no longer have to update `RegisterUser` at the same time.